TP 2 : code du notebook

Code extrait de `TP2_IBM_Model1_canal_bruite_evaluation.ipynb`, dans l'ordre des cellules (sans les sorties).

0. Installation

```
# INSTALL
# Importe les bibliothèques de base (le pip est commenté : tout est déjà dans le venv).
#!pip -q install sentence-transformers sacrebleu unbabel-comet nltk datasets
import re, math, random
from collections import defaultdict, Counter
import numpy as np
random.seed(0)
```

1. Charger votre corpus

```
# CHARGEMENT
# Lit yi.txt et fr.txt bruts et les découpe en phrases avec wtpsplit -> phrases["yi"],
phrases["fr"].
# wtpsplit est listé dans requirements.txt

import os
from wtpsplit import SaT

sat = SaT("sat-3l-sm")  # petit modèle, suffisant pour ce TP

def load_lines(path, toy):
    if os.path.exists(path):
        # encoding="utf-8-sig" permet de supprimer le caractère BOM
        with open(path, encoding="utf-8-sig") as f:
            return [l.strip() for l in f if l.strip()]
    print(f"[info] {path} introuvable — corpus-jouet utilisé.")
    return toy

# textes bruts
yi_lines = load_lines("yi.txt", [])
fr_lines = load_lines("fr.txt", [])

phrases = {}
# Les résultats seront gardés dans phrases["yi"] et phrases["fr"]
for name, lines in [("yi", yi_lines), ("fr", fr_lines)]:
    phrases[name] = [s.strip()
        # On saute les titres
        for l in lines if not l.startswith("#")
        # On supprime les * de mise en forme
        for s in sat.split(l.replace("*", ""))]

    print(f"{name}: {len(lines)} lignes -> {len(phrases[name])} phrases")

    # Afficher 3 phrasese par langue
    for s in phrases[name][:3]:
        print("   ", s)
```

2. Alignement des phrases — à vous de jouer

```python
# ALIGNEMENT 1 : alignement des paragraphes
# Retrait de la mise en forme avec "*" + omission des titres (qui commencent par "#"")
yi_par = [l.replace("*", "") for l in yi_lines if not l.startswith("#")]
fr_par = [l.replace("*", "") for l in fr_lines if not l.startswith("#")]
yi_par = [p for p in yi_par if p]   # retire les séparateurs "***" devenus vides
fr_par = [p for p in fr_par if p]
print(len(yi_par), "paragraphes yiddish /", len(fr_par), "paragraphes français")
```

```python
# ALIGNEMENT 2 : calcul des vecteurs LaBSE de chaque paragraphe
# LaBSE transforme chaque paragraphe en vecteurs
# -> permettra de comparer le yiddish et le français sans traduire

from sentence_transformers import SentenceTransformer

embedder = SentenceTransformer("sentence-transformers/LaBSE")

def embed(sents):
    # normalize_embeddings=True ramène chaque vecteur à la longueur 1
    # Les vecteurs doivent être normalisés pour obtenir leur similarité cosinus (en 3.)
    # show_progress_bar=True affiche une barre de progression
    return embedder.encode(sents, normalize_embeddings=True, show_progress_bar=True)

emb_yi = embed(yi_par)
emb_fr = embed(fr_par)
print(emb_yi.shape, emb_fr.shape)

# Résultats obtenus :
# (466, 768) (474, 768)
```

```python
# ALIGNEMENT 3 : calcul de la matrice de similarité
# Calcul de la similarité cosinus pour comparer chaque paragraphe yiddish à chaque paragraphe
français (sim[i][j])
sim = emb_yi @ emb_fr.T
print(sim.shape)

# la colonne française la plus proche pour chaque ligne yiddish
meilleur = sim.argmax(axis=1)
print("10 premiers paragraphes yiddish -> français :", meilleur[:10])
print("similarité moyenne du meilleur candidat :", sim.max(axis=1).mean().round(2))
print("écart médian de position |j - i| :", np.median(np.abs(meilleur - np.arange(len(meilleur)))))

# Résultats obtenus :
# (466, 474)
# - similarité moyenne du meilleur candidat : 0,76
# - écart médian de position |j - i| : 4.0 (10 paragraphes supplémentaires en français)
```

```python
# ALIGNEMENT 4 : meilleur chemin
# chaque paire formée ajoute sa similarité
# chaque paragraphe sauté, côté yiddish ou côté français, retire 0,3 de pénalité (penalite=0.3)
# L'algorithme peut sauter lorsque cela rapporte plus que cela ne coûte
# (p. ex. pour sauter un paragraphe en trop qui décalerait le reste de l'alignement)

def align_dp(sim, penalite=0.3):
    n, m = sim.shape
    dp = np.full((n + 1, m + 1), -np.inf)
    dp[0, 0] = 0
    back = {}
    for a in range(n + 1):
        for b in range(m + 1):
            if a == 0 and b == 0:
                continue
            best, choice = -np.inf, None
            # apparier le paragraphe yiddish a-1 avec le français b-1
```

```
            if a > 0 and b > 0 and dp[a-1, b-1] + sim[a-1, b-1] > best:
                best, choice = dp[a-1, b-1] + sim[a-1, b-1], (a-1, b-1, 'match')
            # sauter un paragraphe yiddish
            if a > 0 and dp[a-1, b] - penalite > best:
                best, choice = dp[a-1, b] - penalite, (a-1, b, 'skip_src')
            # sauter un paragraphe français
            if b > 0 and dp[a, b-1] - penalite > best:
                best, choice = dp[a, b-1] - penalite, (a, b-1, 'skip_tgt')
            dp[a, b] = best
            back[(a, b)] = choice
    # remonter le chemin depuis la dernière case
    path, a, b = [], n, m
    while (a, b) != (0, 0):
        prev = back[(a, b)]
        if prev[2] == 'match':
            path.append((prev[0], prev[1]))
        a, b = prev[0], prev[1]
    return list(reversed(path))

pairs = align_dp(sim)
print(len(pairs), "paires 1-1")
print("yiddish non appariés :", len(yi_par) - len(pairs),
      "| français non appariés :", len(fr_par) - len(pairs))

# Résultats obtenus :
# 460 paires 1-1
# yiddish non appariés : 6 | français non appariés : 14
```

```
# ALIGNEMENT 5 : vérification des paires
import html
from IPython.display import display, HTML

# 1. Vue d'ensemble des scores
scores = [sim[i, j] for i, j in pairs]
print("score moyen :", np.mean(scores).round(2), "| minimum :", np.min(scores).round(2))
print("paires sous 0,5 :", sum(s < 0.5 for s in scores), "sur", len(pairs))

# 2. Quels paragraphes ont été sautés ?
yi_pris = {i for i, _ in pairs}
fr_pris = {j for _, j in pairs}
yi_saute = [i for i in range(len(yi_par)) if i not in yi_pris]
fr_saute = [j for j in range(len(fr_par)) if j not in fr_pris]
print("yiddish sautés :", yi_saute)
print("français sautés :", fr_saute)

# 3. Les 5 paires les moins sûres, côte à côte
for i, j in sorted(pairs, key=lambda p: sim[p])[:5]:
    print(f"\n--- yi[{i}] <-> fr[{j}]  (score {sim[i, j]:.2f})")
    display(HTML(f"<p dir='rtl' style='text-align:right'>{html.escape(yi_par[i][:200])}</p>"))
    print(fr_par[j][:200])


print("=== Français sautés ===")
for j in fr_saute:
    print(f"fr[{j}] : {fr_par[j][:120]}")

print()
print("=== Yiddish sautés ===")
for i in yi_saute:
    print(f"yi[{i}] :")
    display(HTML(f"<p dir='rtl' style='text-align:right'>{html.escape(yi_par[i][:120])}</p>"))

# Résultats obtenus :
# score moyen : 0.76 | minimum : 0.25
# paires sous 0,5 : 10 sur 460
```

```
# ALIGNEMENT 6 : écriture de src.txt (yiddish) et tgt.txt (français)
src_aligne = [yi_par[i] for i, j in pairs]
```

```
tgt_aligne = [fr_par[j] for i, j in pairs]

# une paire par ligne (= pas de retour à la ligne au sein d'un paragraphe)
assert all("\n" not in p for p in src_aligne + tgt_aligne)
assert len(src_aligne) == len(tgt_aligne)

with open("src.txt", "w", encoding="utf-8") as f:
    f.write("\n".join(src_aligne) + "\n")
with open("tgt.txt", "w", encoding="utf-8") as f:
    f.write("\n".join(tgt_aligne) + "\n")

# Reconstruction de src_sents / tgt_sents
src_sents, tgt_sents = src_aligne, tgt_aligne
print(len(src_aligne), "paires écrites dans src.txt et tgt.txt")
```

```
# RECHARGEMENT & JEU DE TEST
# Recharge src.txt et tgt.txt dans src_sents / tgt_sents et vérifie qu'ils sont alignés.
# toy seront utilisés comme fichiers src et tgt si les fichiers fournis ne pointent nulle part.

toy_src = [
    "das Haus ist groß", "das Buch ist alt", "die Katze schläft", "der Hund läuft",
    "das Kind liest das Buch", "die Frau kauft das Haus", "der Mann liest die Zeitung",
    "das Mädchen sieht die Katze", "der Hund sieht das Kind", "die Frau liest das Buch",
]
toy_tgt = [
    "la maison est grande", "le livre est vieux", "le chat dort", "le chien court",
    "l'enfant lit le livre", "la femme achète la maison", "l'homme lit le journal",
    "la fille voit le chat", "le chien voit l'enfant", "la femme lit le livre",
]

src_sents = load_lines("src.txt", toy_src)
tgt_sents = load_lines("tgt.txt", toy_tgt)

assert len(src_sents) == len(tgt_sents), "src et tgt doivent avoir le même nombre de lignes :
alignez-les d'abord (section 2) !"
print(len(src_sents), "paires de phrases chargées.")


# Jeu de test mis à l'écart de l'entraînement
N_TEST = 30
random.seed(0)   # tirage reproductible

# candidats : paires de 5 à 30 mots dans les deux langues
# candidats = les numéros k des paires (yiddish, français) qui peuvent servir de test
candidats = [k for k in range(len(src_sents))                # k parcourt les numéros de toutes les
paires
             if 5 <= len(src_sents[k].split()) <= 30         # phrase yiddish de 5 à 30 mots
(split() compte les mots séparés par des espaces)
             and 5 <= len(tgt_sents[k].split()) <= 30]       # ET phrase française de 5 à 30 mots

test_idx = set(random.sample(candidats, N_TEST))

test_src = [src_sents[k] for k in sorted(test_idx)]
test_ref = [tgt_sents[k] for k in sorted(test_idx)]

# l'entraînement ne voit plus ces paires
src_sents = [s for k, s in enumerate(src_sents) if k not in test_idx]
tgt_sents = [s for k, s in enumerate(tgt_sents) if k not in test_idx]
print(len(src_sents), "paires d'entraînement /", len(test_src), "paires de test")

# Résultat obtenu :
# "460 paires de phrases chargées.
# 430 paires d'entraînement / 30 paires de test"
```

3. Tokenisation et vocabulaire

```
# TOKENISATION

def tokenize(s):
    # split() coupe uniquement aux espaces
    #return s.lower().split()

    s = s.lower()

    # points de suspension
    s = re.sub(r"\.{2,}", "…", s)

    # élisions
    s = re.sub(r"\b(l|d|j|t|m|s|n|c|qu|jusqu|lorsqu|puisqu)(['’])(?=\w)", r"\1\2 ", s)

    # ponctuation : de chaque côté de chaque signe
    s = re.sub(r"([.,;:!?…«»()\[\]\"“”„—])", r" \1 ", s)
    return s.split()

src_tok = [tokenize(s) for s in src_sents]
tgt_tok = [tokenize(s) for s in tgt_sents]

src_vocab = sorted(set(w for s in src_tok for w in s))
tgt_vocab = sorted(set(w for s in tgt_tok for w in s)) + ["<NULL>"]

print(f"{len(src_vocab)} mots source, {len(tgt_vocab)} mots cible (NULL compris).")
print(src_tok[0], "<->", tgt_tok[0])

# Résultats obtenu après correction :
# "2618 mots source, 2721 mots cible (NULL compris)."
```

```
# Vérification de la tokenisation (français)
tokenize("Souviens-toi du conseil d’un aîné : les mots, l’enfant... qu’il vienne !")
```

```
# Vérification de la tokenisation (yiddish)
tokenize("וויסל איך, ליבער מענטש, וועל שוין לאַנג ניט צייַן, וועל גיין באַגעגענען די צוויי מיידלעך... גיט אַ האַנט")
```

Résultat obtenu (français) :

Résultat obtenu (yiddish) :

```
from collections import Counter

for nom, tok, vocab in [("yiddish", src_tok, src_vocab), ("français", tgt_tok, tgt_vocab)]:
    avec_ponct = [w for w in vocab if w[-1] in ".,;:!?»…)"]
    print(f"{nom} : {len(vocab)} mots distincts, dont {len(avec_ponct)} se terminent par de la
ponctuation")
    print("   exemples :", avec_ponct[:8])
    print("   plus fréquents :", Counter(w for s in tok for w in s).most_common(10))
```

Résultats obtenus avant correction :

Résultats obtenus après correction :

4. IBM Model 1 : construire le modèle de traduction

Un exemple numérique avant le code

```
# Prends une paire de phrases : une phrase yiddish et sa traduction française
# Pour un mot yiddish f, on ne sait pas quel est le mot français correspondant.
# IBM Model 1 répartit une unité de certitude entre tous les mots français de la phrase
# plus un mot français e est susceptible de produire f, plus il reçoit de part

def posteriors_for_word(f, e_candidats, t):
    """
    f           : un mot source (ex. 'das')
    e_candidats : la liste des mots cible de la phrase, NULL compris (ex. ['<NULL>', 'la',
'maison'])
    t           : le dictionnaire courant des t(f|e)

    Retourne un dictionnaire {e: probabilité} qui doit sommer à 1.
    """
    denominateur = sum(t[(f, e)] for e in e_candidats)  # <- fourni, c'est la somme du bas de la
formule

    probas = {}
    for e in e_candidats:
        probas[e] = t[(f, e)] / denominateur

    return probas
```

```
# Testez votre fonction ici avant de continuer : avec t uniforme, vous devez obtenir 0,5 / 0,5.
t_test = defaultdict(lambda: 1.0 / 3)
resultat = posteriors_for_word("das", ["la", "maison"], t_test)
print(resultat)
assert abs(sum(resultat.values()) - 1.0) < 1e-6, "Les probabilités doivent sommer à 1."
assert abs(resultat["la"] - 0.5) < 1e-6, "Avec t uniforme, on doit retrouver 0,5 / 0,5."
print("Test réussi !")
```

```
def e_step(src_tok, tgt_tok, t):
    """Parcourt tout le corpus et accumule les comptes fractionnaires, en
    appelant posteriors_for_word() pour chaque mot source rencontré."""
    count = defaultdict(float)
    total_e = defaultdict(float)
    for f_sent, e_sent_raw in zip(src_tok, tgt_tok):
        e_sent = ["<NULL>"] + e_sent_raw
        for f in f_sent:
            probas = posteriors_for_word(f, e_sent, t)
            for e, p in probas.items():
                count[(f, e)] += p
                total_e[e] += p
    return count, total_e

def m_step(count, total_e):
    """Ré-estime t(f|e) = compte(f,e) / total(e). Une paire jamais vue ensemble reste à 0."""
    new_t = defaultdict(float)
    for (f, e), c in count.items():
        new_t[(f, e)] = c / total_e[e]
    return new_t
```

5. Entraînement : faites tourner EM (Expectation-Optimisation)

```
# Initialisation : t(f|e) uniforme sur tout le vocabulaire source, pour un vrai entraînement
t = defaultdict(lambda: 1.0 / len(src_vocab))

N_ITER = 10
for it in range(N_ITER):
    count, total_e = e_step(src_tok, tgt_tok, t)
    t = m_step(count, total_e)

# petit test de cohérence : la somme des t(f|e) sur f, pour un e fixé, doit faire ~1
s = sum(t[(f, tgt_vocab[0])] for f in src_vocab)
print(f"Somme de contrôle pour e={tgt_vocab[0]!r} (doit être proche de 1) : {s:.3f}")
```

6. Sous-produit utile : extraire un lexique bilingue

```
def top_translation(e, k=3):
    scores = [(t[(f, e)], f) for f in src_vocab]
    scores.sort(reverse=True)
    return scores[:k]

for e in tgt_vocab:
    if e == "<NULL>":
        continue
    best = top_translation(e)
    print(f"{e:12s} -> " + ", ".join(f"{f} ({p:.2f})" for p, f in best))
```

7. Modèle de langue (fourni)

```
class BigramLM:
    def __init__(self, sentences):
        self.unigrams = Counter()
        self.bigrams = Counter()
        self.vocab = set()
        for s in sentences:
            toks = ["<s>"] + tokenize(s) + ["</s>"]
            self.vocab.update(toks)
            for w in toks:
                self.unigrams[w] += 1
            for w1, w2 in zip(toks, toks[1:]):
                self.bigrams[(w1, w2)] += 1
        self.V = len(self.vocab)

    def logprob(self, sentence_tokens):
        toks = ["<s>"] + sentence_tokens + ["</s>"]
        lp = 0.0
        for w1, w2 in zip(toks, toks[1:]):
            num = self.bigrams[(w1, w2)] + 1
            den = self.unigrams[w1] + self.V
            lp += math.log(num / den)
        return lp

lm = BigramLM(tgt_sents)
print("logP('le chat dort') =", lm.logprob(tokenize("le chat dort")))
```

8. Le décodeur : combiner $P(f \mid e)$ et $P(e)$

```python
import itertools, random

def candidate_words(f, k=3):
    """Les k meilleures traductions candidates d'un mot source, d'après t(f|e). Fourni."""
    scores = [(t[(f, e)], e) for e in tgt_vocab if e != "<NULL>"]
    scores.sort(reverse=True)
    return [e for _, e in scores[:k]]

def decode_greedy(f_sent_tokens):
    """Traduction mot-à-mot gloutonne : pour chaque mot source, prend sa meilleure traduction.
Fourni."""
    return " ".join(candidate_words(f, k=1)[0] for f in f_sent_tokens)
```

À vous : noter une combinaison candidate

```python
def score_combo(f_sent_tokens, combo):
    """
    f_sent_tokens : les mots source, ex. ['die', 'katze', 'läuft']
    combo         : une combinaison candidate de mots cible, même longueur, ex. ('le', 'chat',
'court')

    Retourne le score du canal bruité (un seul nombre, plus il est grand, mieux c'est).
    """
    score_traduction = sum(math.log(t[(f, e)] + 1e-12) for f, e in zip(f_sent_tokens, combo))
    # Indice : zip(f_sent_tokens, combo) donne les paires (f_j, e_j) dans l'ordre.
    # Indice 2 : utilisez math.log(t[(f, e)] + 1e-12) -- le +1e-12 évite log(0).

    score_langue = lm.logprob(list(combo))  # <- fourni

    return score_traduction + score_langue
```

```python
def decode_noisy_channel(f_sent_tokens, k=3, n_samples=200):
    """Génère des combinaisons candidates (produit cartésien si petit, sinon tirage aléatoire),
    et renvoie celle qui maximise score_combo(). Fourni."""
    cand_lists = [candidate_words(f, k=k) for f in f_sent_tokens]

    total_combos = 1
    for c in cand_lists:
        total_combos *= len(c)

    if total_combos <= n_samples:
        combos = itertools.product(*cand_lists)
    else:
        combos = (tuple(random.choice(c) for c in cand_lists) for _ in range(n_samples))

    best_score, best_combo = -1e18, None
    for combo in combos:
        s = score_combo(f_sent_tokens, combo)
        if s > best_score:
            best_score, best_combo = s, combo

    return " ".join(best_combo) if best_combo else ""

test_sent = tokenize("die Katze läuft")
print("Mot-à-mot :", decode_greedy(test_sent))
print("Canal bruité :", decode_noisy_channel(test_sent))
```

9. Traduire un jeu de test

```
# Séparez quelques phrases de test (idéalement non vues à l'entraînement, sur votre vrai corpus).
#test_src = src_sents[-3:]
#test_ref = tgt_sents[-3:]

system_hyps = [decode_noisy_channel(tokenize(s)) or decode_greedy(tokenize(s)) for s in test_src]

for s, h, r in zip(test_src, system_hyps, test_ref):
    print(f"SRC: {s}\nHYP: {h}\nREF: {r}\n")
```

10. Évaluation — votre système vs. l'état de l'art

```
import sacrebleu

baseline_hyps = [decode_greedy(tokenize(s)) for s in test_src]

def show_bleu(name, hyps, refs):
    bleu = sacrebleu.corpus_bleu(hyps, [refs])
    print(f"{name:25s} sacreBLEU = {bleu.score:.1f}")
    return bleu.score

show_bleu("Mot-à-mot (baseline)", baseline_hyps, test_ref)
show_bleu("IBM Model 1 + canal bruité", system_hyps, test_ref)
```

Comparaison avec un système moderne (fourni)

```
# TRANSFORMERS
try:
    from transformers import pipeline
    # Codes de langue NLLB adaptés à notre paire : yiddish ("ydd_Hebr") -> français ("fra_Latn")
    translator = pipeline("translation", model="facebook/nllb-200-distilled-600M",
                          src_lang="ydd_Hebr", tgt_lang="fra_Latn")
    nllb_hyps = [translator(s)[0]["translation_text"] for s in test_src]
    hyps_nllb_transformers = nllb_hyps  # gardé pour COMET (nllb_hyps est réécrit par la cellule Torch)
    for s, h in zip(test_src, nllb_hyps):
        print(f"SRC: {s}\nNLLB: {h}\n")
    show_bleu("NLLB-200 (600M)", nllb_hyps, test_ref)
except Exception as e:
    print("NLLB indisponible pour cette paire de langues :", e)
    print("-> utilisez un LLM via prompt (cellule suivante) comme point de comparaison.")

# Score obtenu :
# NLLB-200 (600M)           sacreBLEU = 13.4
```

```
# TORCH
try:
    import torch
    from transformers import AutoTokenizer, AutoModelForSeq2SeqLM
    # Codes de langue NLLB adaptés à notre paire : yiddish ("ydd_Hebr") -> français ("fra_Latn")
    nom = "facebook/nllb-200-distilled-600M"
    tokenizer = AutoTokenizer.from_pretrained(nom, src_lang="ydd_Hebr")
    model = AutoModelForSeq2SeqLM.from_pretrained(nom)
    appareil = "cuda" if torch.cuda.is_available() else "cpu"
    model = model.to(appareil)

    def translator(s):
        entree = tokenizer(s, return_tensors="pt").to(appareil)
        sortie = model.generate(**entree,
                                forced_bos_token_id=tokenizer.convert_tokens_to_ids("fra_Latn"),
```

```
                                    max_new_tokens=256)
        return tokenizer.decode(sortie[0], skip_special_tokens=True)

    nllb_hyps = [translator(s) for s in test_src]
    hyps_nllb_torch = nllb_hyps  # gardé pour COMET
    for s, h in zip(test_src, nllb_hyps):
        print(f"SRC: {s}\nNLLB: {h}\n")
    show_bleu("NLLB-200 (600M)", nllb_hyps, test_ref)
except Exception as e:
    print("NLLB indisponible pour cette paire de langues :", e)
    print("-> utilisez un LLM via prompt (cellule suivante) comme point de comparaison.")

# Score obtenu :
# NLLB-200 (600M)           sacreBLEU = 13.2
```

Alternative / complément : COMET

```
try:
    from comet import download_model, load_from_checkpoint
    model_path = download_model("Unbabel/wmt22-comet-da")
    comet_model = load_from_checkpoint(model_path)

    # COMET est calculé pour chaque système, sur les mêmes 30 phrases de test.
    # Une cellule en échec (ex. pipeline absent avec Transformers 5) laisse sa variable indéfinie :
le système est alors sauté.
    systemes = {
        "Mot-à-mot": baseline_hyps,
        "IBM Model 1 + canal bruité": system_hyps,
        "NLLB-200 (Transformers)": globals().get("hyps_nllb_transformers"),
        "NLLB-200 (Torch)": globals().get("hyps_nllb_torch"),
    }
    for nom, hyps in systemes.items():
        if hyps is None:
            print(f"{nom:28s} COMET non calculé (traductions indisponibles)")
            continue
        data = [{"src": s, "mt": h, "ref": r} for s, h, r in zip(test_src, hyps, test_ref)]
        out = comet_model.predict(data, batch_size=8, gpus=0)
        print(f"{nom:28s} COMET = {out.system_score:.3f}")
except Exception as e:
    print("COMET indisponible :", e)
```